

LLD Case Study 4: Car Rental System (Hertz / Zoomcar)

Target Patterns: Factory Pattern, Strategy Pattern, State Pattern

Key Engineering Focus: Multi-branch vehicle inventory, dynamic pricing algorithms, vehicle lifecycle states, and reservation management.

1. Problem Statement & Functional Requirements

Design an automated Car Rental Management platform operating across multiple cities and airport branches.

Requirements:

- 1. **Vehicle Hierarchy (Factory Pattern):** Manage diverse vehicle classes (Economy Hatchback, Compact SUV, Luxury Sedan, Electric Vehicle) with distinct specifications and rates.
- 2. **Dynamic Pricing (Strategy Pattern):** Calculate rental costs using swappable pricing algorithms (Base Daily Rate, Surge/Peak Season Pricing, Distance/Mileage Model).
- 3. **Reservation State Machine (State Pattern):** Reservations progress from Pending → Confirmed → PickedUp → Completed (or Cancelled).
- 4. **Multi-Location Inventory:** Allow vehicle pickup at one branch and return at a different branch with inter-branch inventory rebalancing.

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

What the model gets right and what it hides

Concern	Status	Detail
Pricing variants	Good	<code>PricingStrategy</code> keeps daily, surge and mileage pricing out of the reservation
Double booking	Missing	<code>Reservation.__init__</code> never checks <code>vehicle.is_available</code> , so the same car can be reserved twice
Future bookings	Missing	A boolean flag cannot say "free Monday, booked Tuesday"; availability must be a date range
Cancellation	Missing	<code>CANCELLED</code> exists but no method reaches it, and the vehicle would stay unavailable
Money as float	Risky	$0.1 + 0.2 \neq 0.3$; bill in <code>Decimal</code> or integer cents and round once
Late return, damage fee	Not modelled	Extra charge after the agreed return time; compute from timestamps, not days alone

Tests: pricing, lifecycle, and the double-booking bug

This block extends the implementation above. The last part adds a `RentalDesk` that owns date-range availability.

Extensions interviewers ask for

1. **Pickup and drop-off at different branches:** add a `Branch` and a location on each `Vehicle`; availability then depends on where the previous rental ended, and a relocation fee becomes a pricing input.
2. **Insurance and add-ons:** wrap the price with the Decorator pattern (`GpsAddOn(InsuranceAddOn(base_price))`) so add-ons compose without subclass explosions.

3. **Concurrent bookings:** the overlap check and the insert must be atomic; in-process use a lock per vehicle, in a database use an exclusion constraint on `(vehicle_id, daterange)` so the database rejects overlaps itself.
4. **Dynamic pricing:** feed utilisation and the date into a `PricingStrategy` ; keep the quoted price on the reservation so later strategy changes never alter an existing booking.

Follow-up questions

- *Why is the overlap test `start < other_end and other_start < end` ?* It treats intervals as half-open `[start, end)` , so a rental ending on day 4 and one starting on day 4 do not clash.
- *Where does a lock on the whole fleet hurt?* Every booking serialises behind every other one; locking per vehicle (or per vehicle-and-date bucket) keeps unrelated bookings parallel.
- *How do you price a late return?* Store the due timestamp, charge `ceil(hours_late)` times an hourly rate beyond a grace period, and add it as a separate line on the bill.